

POLYTECH NICE SOPHIA
Département Robotique Autonome
930 Rte des Colles, 06410 Biot

RAPPORT DE STAGE DE FIN DE 4^{EME} ANNÉE

[INSERER LE TITRE DU STAGE ICI]

Présenté par : **Thomas CHAVAROCHE**

Entreprise d'accueil : Polytech'Lab

Adresse Entreprise : Campus SophiaTech, 930 Route des Colles, 06903 SOPHIA
ANTIPOLIS CEDEX

Responsable de stage (Entreprise) : Pascal MASSON, Fonction : f

Tuteur enseignant (Polytech) : Delphine CALLAGHAN, Directrice Adjointe - Vie
Etudiante et Communication - Référente Racisme, Antisémitisme et Discriminations -
Professeur d'anglais

Année Universitaire : [2026 - 2027]

[Répéter le Titre du Stage ici]

RAPPORT DE STAGE DE FIN D'ÉTUDES

Prénom NOM

Remerciements

Je tiens à remercier tout d'abords Monsieur Pascal Masson pour cette oportunity de stage. Trvailler sur ROS et en particulier sur ces robots est une excellente oportunity pour moi. Je voudrais aussi remercier Monsieur Frédéric Juan pour l'idée de stage, l'aménagement des salles et l'aide apporté. Finalement, notre manager Monsieur Matthias Vermot-Desroches, qui nous aura aider a mener a bien nos projets dans les meilleurs conditions qui soient.

Table des matières

Remerciements	3
Table des annexes	6
Glossaire	7
Glossaire	7
1 Introduction Générale	8
1.1 Contexte et Problématique	8
1.2 Objectifs du Projet	8
1.3 Structure du Document	8
2 PRISE EN MAIN, SLAM ET NAVIGATION AUTONOME	10
2.1 Prise en main de la plateforme, SLAM et Navigation Autonome	10
2.1.1 Contextualisation et environnement de travail	10
2.1.2 Structure du Workspace ROS 2	11
2.1.3 Définition et Test d'une Navigation Autonome Réactive	11
2.1.4 SLAM 2D et Navigation Autonome avec Nav2	12
2.1.5 Cartographie 3D et Navigation avec RTAB-Map	13
2.1.6 Positionnement Méthodologique et Bilan	14
3 TEST ET MISE REALISATION D'UN SUIVEUR DE LIGNES	15
3.1 Partie 2 : Développer le Suiveur de Ligne Autonome	15
3.1.1 Phase 1 : Tests de Fonctionnement, Évaluation Matérielle et Développement des TP	15
3.1.2 Phase 2 : Analyse Visuelle Avancée, Architecture Découplée et Régulation Adaptative	19
3.1.3 Synthèse de la Phase 3 : Fusion Perception, Contrôle & Sécurité . .	22
3.1.4 1. Nœud de Perception & FSM : <code>vision_line_node</code>	22
3.1.5 2. Nœud de Contrôle Dynamique : <code>control_line_node</code>	23
3.1.6 3. Filtre de Sécurité 3D : <code>safety_gate_3d_node</code>	23
3.1.7 Lien avec l'Architecture Globale du Projet	23
3.2 Aparte : Intégration Globale et Validation Système	24
4 Contributions Transversales, Documentation & Environnement de Test	25
4.0.1 1. Rédaction de la Documentation Technique : Dépannage Docker .	25
4.0.2 2. Exploration Fonctionnelle & Préparation du Circuit d'Essai . . .	25
4.0.3 3. Évaluation en Conditions Réelles & Contraintes Environnementales	26
4.0.4 4. Proximité et Synergie avec le FabLab	26

5 Conclusion et Perspectives	27
5.1 Conclusion Générale	27
5.2 Perspectives et Évolutions Futures	27
Annexes	29
Annexe A : Fiche Technique — Dépannage Docker	29
Annexe B : Dépôt de Code Source & Ressources (GitHub)	30

Table des annexes

La liste des annexes ci-dessous respecte leur ordre d'apparition dans le présent document :

Annexe A : [Fiche Technique — Dépannage Docker]	Page [29]
Annexe B : [Dépôt de Code Source & Ressources (GitHub)]	Page [30]

Glossaire

Note : Les termes techniques définis dans ce glossaire sont signalés par un astérisque () lors de leur première apparition dans le texte.*

CAN (Controller Area Network) Bus de communication série utilisé pour l'échange de données bas niveau entre le PC embarqué (NUC) et les moteurs de la base Scout Mini.

Costmap (Carte de coût) Représentation grille-2D ou 3D de l'environnement utilisée par Nav2* pour évaluer les zones libres, les obstacles et les marges de sécurité (*hitbox*).

FSM (Finite State Machine) Machine à États Finis. Modèle de programmation organisant le comportement du robot sous forme d'états logiques (ex : AVANCER, TOURNER, STOP).

ICP (Iterative Closest Point) Algorithme de recadrage de nuages de points 3D successifs permettant de calculer l'odométrie exacte du robot à partir du LiDAR*.

IMU (Inertial Measurement Unit) Unité de mesure inertielle mesurant les accélérations linéaires et les vitesses angulaires (gyroscope) du robot.

LiDAR (Light Detection and Ranging) Capteur de télédétection par laser fournissant une carte de points tridimensionnelle (PointCloud2) de l'environnement.

Nav2 Suite logicielle officielle de navigation sous ROS* 2 assurant la planification de trajectoire globale et le contrôle local d'évitement d'obstacles.

noVNC Interface web permettant d'accéder à l'affichage graphique du conteneur Docker (RViz, fenêtres OpenCV) via un navigateur.

PID (Proportionnel Intégral Dérivé) Algorithme de régulation en boucle fermée calculant la commande moteur en fonction de l'erreur spatiale instantanée et de sa variation.

ROS 2 (Robot Operating System 2) Framework logiciel modulaire pour la robotique, basé sur une architecture de communication par publication/abonnement de topics via DDS.

ROI (Region of Interest) Zone spécifique découpée dans l'image de la caméra dans laquelle sont appliqués les traitements visuels (ex : scanlines).

RTAB-Map (Real-Time Appearance-Based Mapping) Solution de SLAM* 3D basée sur la fermeture de boucle visuelle et le recadrage LiDAR ICP*.

SLAM (Simultaneous Localization and Mapping) Cartographie et Localisation Simultanées. Technique permettant à un robot de construire la carte d'un environnement inconnu tout en s'y localisant.

Chapitre 1

Introduction Générale

1.1 Contexte et Problématique

La robotique mobile autonome connaît un essor majeur dans les milieux industriels et urbains, nécessitant des systèmes capables de combiner la planification globale de trajectoire, le suivi précis de voies au sol et la réactivité face aux imprévus. Dans le cadre de ce projet mené à Polytech Nice-Sophia, l'objectif principal est d'exploiter la plateforme robotique mobile **AgileX Scout Mini** afin de mettre en place une chaîne de navigation routière autonome complète sous **ROS 2 Humble**.

Le passage à ROS 2, combiné à un environnement conteneurisé **Docker**, offre des garanties de modularité et de portabilité matérielle, mais pose également des défis d'intégration système, de gestion mémoire et de synchronisation multi-capteurs temps réel.

1.2 Objectifs du Projet

Les travaux confiés au sein de l'équipe visaient à répondre à un triple objectif :

- **Valider la pile de navigation de base** : Assurer la cartographie 2D/3D et la localisation simultanée (SLAM*) pour permettre au robot de se déplacer de manière autonome dans un environnement encombré.
- **Concevoir un cadre pédagogique et algorithmique** : Élaborer un support de Travaux Pratiques (TP 1 à 5) pour les promotions futures, tout en développant un algorithme de suivi de ligne visuel performant capable de faire face aux aléas d'éclairage et aux virages serrés.
- **Garantir la fusion et la sécurité tridimensionnelle** : Développer la brique de suivi de voie et de sécurité 3D (*Safety Gate*) s'intégrant au sein d'une architecture globale contrôlée par un Manager IA interceptant la signalisation routière.

1.3 Structure du Document

Afin de rendre compte de la progression de ces travaux, ce rapport s'organise en trois chapitres principaux :

- **Le Chapitre 1** présente l'environnement de développement Docker/ROS 2, la prise en main du robot Scout Mini et le déploiement des stacks de cartographie SLAM 2D (`slam_toolbox`) et 3D (**RTAB-Map**).
- **Le Chapitre 2** détaille la démarche itérative de réalisation du suiveur de ligne : de la création des exercices pédagogiques basés sur l'odométrie et le LiDAR, jusqu'à la conception de l'architecture découplée Perception/Contrôle et le chan-

gement stratégique de capteur optique. Jusqu'à la fusion multi-capteurs (Vision + IMU* + LiDAR 3D), la validation de l'intégration avec la machine à état et l'IA de détection de panneau de signalisation.

- **Le Chapitre 3** expose les contributions transversales (documentation de dépannage Docker, aménagement de la piste et synergie avec le FabLab).
- **Enfin, la Conclusion et les Perspectives** synthétisent les résultats obtenus et proposent des axes d'évolution tant sur le plan algorithmique que matériel.

Chapitre 2

PRISE EN MAIN, SLAM ET NAVIGATION AUTONOME

2.1 Prise en main de la plateforme, SLAM et Navigation Autonome

2.1.1 Contextualisation et environnement de travail

Matériel et Architecture Embarquée

Le robot utilisé est basé sur une plateforme mobile (Scout Mini / Bunker Mini) équipée d'un ensemble de capteurs et d'unités de calcul :

- **Unité de calcul** : Asus NUC 15 Pro embarquant ROS 2 Humble sous environnement conteneurisé Docker.
- **Capteurs principaux** : LiDAR 3D Robosense Helios 16P, caméra RGB-D Intel RealSense D435, IMU Phidgetspatial, GPS RTK simpleRTK2B et un routeur Teltonika RUT956 pour les communications réseau.

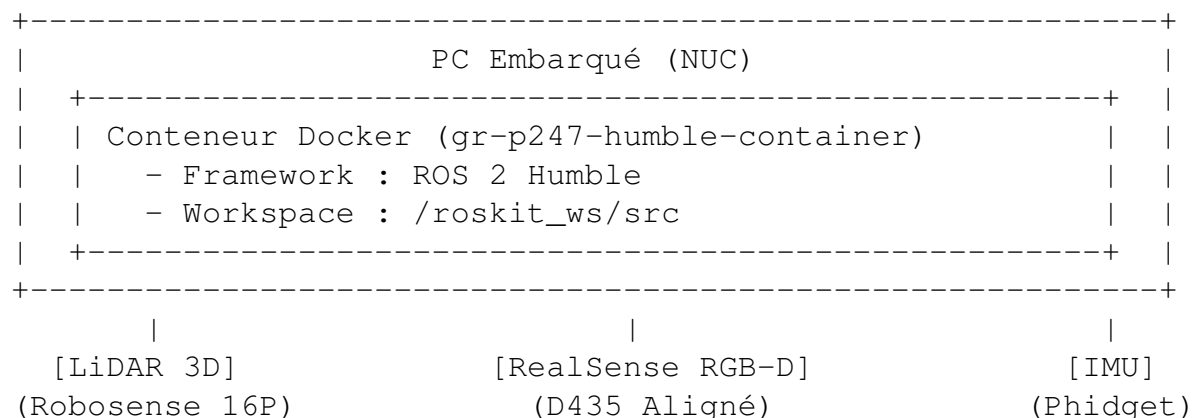


FIGURE 2.1 – Architecture matérielle et logicielle embarquée du robot.

Le contrôle manuel s'effectue via une télécommande radio AgileX, où le commutateur n°2 permet de basculer le Scout Mini du mode télécommandé au mode autonome (communication CAN* bus avec le PC embarqué). La connexion au robot s'effectue en Wi-Fi via SSH ou interface web noVNC*.

Prise en main de ROS 2 et de l'environnement Docker

Venant d'un environnement ROS 1, la transition vers ROS 2 a nécessité un temps d'adaptation aux nouveaux outils et concepts :

- **Commandes et compilation** : Remplacement de `catkin_make` par `colcon build` (en limitant les workers à 1 via `-parallel-workers 1` pour éviter la saturation mémoire lors de la compilation de packages lourds comme Nav2* ou RTAB-Map).
- **Outillage de diagnostic** : Prise en main des réflexes d'inspection des flux (`ros2 topic list`, `ros2 topic echo`, `ros2 topic hz`).
- **Gestion du conteneur Docker** : L'utilisation de Docker a présenté plusieurs défis opérationnels :
 - La persistance des configurations et des dépendances lors de la reconstruction de l'image. Pour éviter de ressourcer manuellement l'environnement à chaque ouverture de session Bash, le fichier `~/ .bashrc` du conteneur a été configuré pour inclure automatiquement `source /opt/ros/humble/setup.bash` et `source /roskit_ws/install/setup.bash`.
 - L'affichage graphique via noVNC/RViz2 présentait parfois des blocages de rendu ou des flux figés, nécessitant des vérifications systématiques des topics publiés.

2.1.2 Structure du Workspace ROS 2

Le développement logiciel a été réalisé au sein du workspace situé dans `/roskit_ws/src`. La structure globale s'articule comme suit :

```
/roskit_ws/src/  
gr_bringup/      # Packages de lancement hardware  
                  # et configuration des capteurs  
  launch/  
    main.launch.py      # Lancement global des  
                        # nodes capteurs  
    component_realsense.launch.py # Config camra (alignement  
                                # depth/RGB)  
navigation2/      # Suite Nav2 officielle (branche Humble)  
  nav2_bringup/  
    params/nav2_params.yaml # Config costmaps*,  
                            # planners et controllers  
rtabmap_ros/      # Package d'intgration RTAB-Map pour ROS 2  
slam_toolbox/     # Algorithmes de cartographie 2D
```

2.1.3 Définition et Test d'une Navigation Autonome Réactive

Avant de déployer des stacks complexes (Nav2, SLAM), une première étape a consisté à concevoir un nœud Python réactif basique (`robot_control`) permettant de valider la chaîne d'actionneurs et de capteurs.

Architecture du Nœud de Contrôle

Le nœud repose sur trois composantes ROS 2 :

1. **Publisher (/cmd_vel)** : Envoi de messages `geometry_msgs/msg/Twist` à une fréquence régulière pour commander les vitesses linéaires et angulaires du

robot.

2. Subscribers & Callbacks :

- `/odom` (`nav_msgs/msg/Odometry`) : Récupération de la position et extraction de l'angle de cap (*Yaw*) par conversion du quaternion d'orientation.
- `/rslidar_points` (`sensor_msgs/msg/PointCloud2`) : Filtrage des points 3D du LiDAR en hauteur pour isoler les obstacles situés à hauteur du châssis.

3. **Timer (10 Hz)** : Exécution continue d'une boucle de contrôle afin d'éviter le déclenchement du « watchdog » de sécurité du Scout Mini en cas de perte de signal.

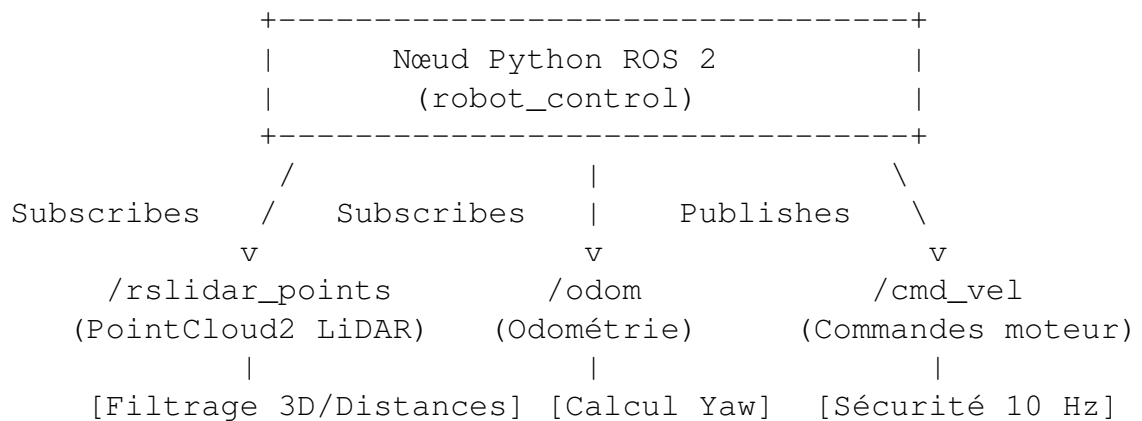


FIGURE 2.2 – Flux de données du nœud de contrôle réactif.

2.1.4 SLAM 2D et Navigation Autonome avec Nav2

L'objectif suivant était d'assurer la cartographie et le déplacement autonome dans les couloirs du bâtiment.

Conversion Point Cloud vers LaserScan

Nav2 et `slam_toolbox` exploitent principalement des données 2D (`sensor_msgs/msg/LaserScan`). Le LiDAR Robosense fournissant un nuage de points 3D (`PointCloud2`), le nœud `pointcloud_to_laserscan` a été configuré pour projeter les points situés entre $-0,1$ m et $+0,5$ m sur un plan 2D.

Cartographie avec `slam_toolbox`

Le mapping 2D a été réalisé avec `slam_toolbox` en mode `online_async`. Ce mode permet de privilégier la mise à jour temps réel de la pose du robot par rapport au traitement exhaustif de tous les scans du LiDAR.

Adaptation aux Contraintes Physiques : Dimensionnement de la Hitbox

L'une des difficultés majeures rencontrées lors des essais physiques résidait dans **l'envergure du robot par rapport à la taille réduite de la salle d'expérimentation**.

Par défaut, les zones d'inflation des cartes de coût (*costmaps*) dans Nav2 empêchaient le robot de planifier des trajectoires dans des passages étroits ou près des portes,

considérant le robot comme « bloqué » par ses propres bordures de sécurité. Pour résoudre ce problème :

- Le paramètre `robot_radius` a été réduit à 0,35 m dans `nav2_params.yaml`.
- Le paramètre `inflation_radius` a été abaissé à 0,55 m.

Cette modification de la « hitbox » virtuelle a permis au robot de naviguer dans la salle et de franchir les portes du bâtiment tout en maintenant un évitement d'obstacle fonctionnel.

2.1.5 Cartographie 3D et Navigation avec RTAB-Map

Pour offrir un retour visuel enrichi et une meilleure robustesse, la stack SLAM 3D basée sur **RTAB-Map** a été déployée.

Configuration et Synchronisation des Capteurs

RTAB-Map combine les données de la caméra Intel RealSense D435 et du LiDAR 3D. Pour garantir un fonctionnement correct :

- **Alignement d'image** : Le paramètre `align_depth.enable:=true` a été injecté dans le fichier `component_realsense.launch.py` afin d'aligner la matrice de profondeur sur le flux RGB.
- **Tolérance temporelle** : Pour faire face aux légers décalages d'émission entre capteurs, l'option `approx_sync:=true` a été activée avec un intervalle `approx_sync_max_interval:=0.2`.
- **Odométrie ICP*** : Plutôt que de s'appuyer uniquement sur l'odométrie visuelle (sensible aux variations d'éclairage), l'odométrie par algorithme **ICP (*Iterative Closest Point*)** basée sur le nuage de points LiDAR 3D a été privilégiée pour sa précision dans les espaces intérieurs.

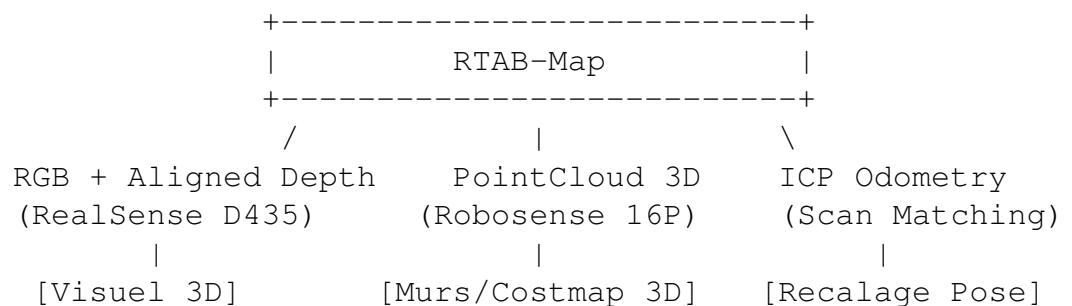


FIGURE 2.3 – Architecture d'intégration capteurs avec RTAB-Map.

Intégration Costmap 3D dans Nav2

Afin que Nav2 prenne en compte la carte 3D générée par RTAB-Map sans ré-interposer de filtre 2D réducteur, la couche d'obstacle (`ObstacleLayer`) du fichier `nav2_params.yaml` a été directement reliée au topic `/rslidar_points` :

Listing 2.1 – Configuration de l'ObstacleLayer dans Nav2 pour nuage de points 3D.

```
obstacle_layer:  
  plugin: "nav2_costmap_2d::ObstacleLayer"  
  enabled: true  
  observation_sources: pointcloud  
  pointcloud:  
    topic: /rslidar_points  
    max_obstacle_height: 2.0  
    min_obstacle_height: 0.1  
    clearing: true  
    marking: true  
    data_type: "PointCloud2"
```

Cela a permis d'exploiter directement la totalité du nuage de points pour la détection et le contournement dynamique des obstacles en temps réel.

2.1.6 Positionnement Méthodologique et Bilan

Le travail réalisé au cours de ces 3 premières semaines a permis de valider l'ensemble de la chaîne de navigation du robot.

Compte tenu du calendrier global du projet (durée totale de 4 mois) et de l'ampleur de la **Partie 2** (qui constitue le cœur de la mission principale), le choix a été fait de traiter la théorie sous-jacente des algorithmes de SLAM (graphes d'optimisation, filtres de Kalman, matrices de covariance ICP) d'un point de vue **pragmatique et appliqué**. L'objectif prioritaire était de disposer d'une plateforme autonome opérationnelle, d'un environnement Docker maîtrisé et d'un retour visuel 3D fiable pour pouvoir concentrer les efforts sur la suite des développements.

Chapitre 3

TEST ET MISE REALISATION D'UN SUIVEUR DE LIGNES

3.1 Partie 2 : Développer le Suiveur de Ligne Autonome

Cette partie retrace l'ensemble des travaux menés sur une durée de deux mois et demi pour réaliser un robot autonome capable de suivre des lignes, de respecter la signalisation routière et de s'adapter à son environnement. Afin d'assurer une montée en compétences progressive et de construire une base solide pour de futurs Travaux Pratiques (TP) destinés aux étudiants, les développements ont été structurés en plusieurs phases successives.

3.1.1 Phase 1 : Tests de Fonctionnement, Évaluation Matérielle et Développement des TP

Avant d'aborder la détection visuelle des lignes ou la reconnaissance de panneaux, une phase préalable majeure a consisté à tester la réactivité de la plateforme physique (Scout Mini), à évaluer la portée utile des capteurs dans la salle d'expérimentation préparée à cet effet, et à fixer les plages de vitesse de sécurité pour la navigation autonome.

Cette première phase a débouché sur la conception de 5 exercices à difficulté croissante, intégrés au document pédagogique *TP : Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2*.

1. Étape 1 : Prise en main du mouvement par Odométrie (Exercices 1 et 2)

Les deux premiers programmes avaient pour but de valider la boucle de commande basique sur le topic `/cmd_vel` et d'exploiter les retours d'odométrie issus du topic `/odom`.

- **Exercice 1 – Translation simple** : Validation des déplacements rectilignes sur une distance cible définie. La distance parcourue d est calculée en continu par la relation euclidienne :

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2}$$

- **Exercice 2 – Chaînage Translation / Rotation** : Introduction des variables d'état pour dissocier les phases d'avance et de pivotement. Le calcul de l'angle parcouru $\Delta\theta$ réaligne la variation d'orientation dans l'intervalle $[-\pi, \pi]$:

Listing 3.1 – Extrait de la boucle de contrôle pour la translation et rotation (Code Ex 2).

```
def boucle(self):
    msg = Twist()

    if self.en_mouvement:
        if self.distance_parcourue() < self.distance_cible and self
        .avance:
            msg.linear.x = 0.2 # Avance
        elif self.distance_parcourue() < abs(self.distance_cible)
        and not self.avance:
            msg.linear.x = -0.2 # Recule
        else:
            msg.linear.x = 0.0 # Stop
            self.en_mouvement = False
            self.get_logger().info('Arrive !')

    if self.en_rotation:
        if abs(self.angle_parcouru()) < abs(self.target_angle):
            msg.angular.z = 0.4 if self.target_angle > 0 else -0.4
        else:
            msg.angular.z = 0.0
            self.en_rotation = False
            self.get_logger().info('Rotation terminee !')

    self.pub.publish(msg)
```

Ce test préliminaire a permis de constater les limites de l'odométrie pure (dérive glissante au sol) et la nécessité de rafraîchir les commandes à une fréquence de 10 Hz au minimum pour éviter l'arrêt du robot par sécurité (*watchdog* de la base mobile).

2. Étape 2 : Intégration du LiDAR 3D et Machine à États Réactive (Exercices 3 et 4)

L'étape suivante visait à intégrer le LiDAR Robosense Helios 16P pour garantir la sécurité du robot dans la salle d'expérimentation.

- **Filtrage du nuage de points (PointCloud2)** : Le LiDAR étant monté en hauteur sur le châssis, un filtre géométrique sur la composante verticale z (ex : $-0,5\text{ m} < z < 0,2\text{ m}$) a été appliqué via `pc2.read_points()` pour éliminer les retours du sol ainsi que les faux obstacles en hauteur.
- **Machine à États Finis (FSM*)** : Pour éviter les fonctions bloquantes, la boucle de contrôle a été réécrite sous forme de machine à états réactive (AVANCER → TOURNER). Lorsque le LiDAR détecte un obstacle à une distance inférieure au seuil de sécurité dans un couloir virtuel de largeur 2,0 m, le robot interroge la fonction `meilleure_direction()` pour basculer du côté offrant le plus grand espace libre.

Listing 3.2 – Machine a etats et filtrage LiDAR 3D pour l'évitement d'obstacles (Code Ex 4).

```
def lidar_callback(self, msg):
    dist_devant = float('inf')
    dist_gauche = float('inf')
    dist_droite = float('inf')
```

```

    for point in pc2.read_points(msg, field_names=('x', 'y', 'z'),
                                skip_nans=True):
        x, y, z = point
        # Filtre hauteur adapte au LiDAR fixe sur le chassis
        if z < -0.5 or z > 0.2:
            continue

        dist = math.sqrt(x**2 + y**2)
        if x > 0.2 and abs(y) < 2.0:
            dist_devant = min(dist_devant, dist)
        if y > 0.2 and abs(x) < 2.0:
            dist_gauche = min(dist_gauche, dist)
        if y < -0.2 and abs(x) < 2.0:
            dist_droite = min(dist_droite, dist)

        self.obstacle_devant = dist_devant < self.distance_securite

def boucle(self):
    msg = Twist()
    if self.etat == "AVANCER":
        if self.obstacle_devant:
            direction = self.meilleure_direction()
            angle = 30 if direction == 'gauche' else -30
            self.rotation_cible = math.radians(angle)
            self.start_yaw = self.yaw
            self.etat = "TOURNER"
        else:
            msg.linear.x = 0.2

    elif self.etat == "TOURNER":
        msg.linear.x = 0.0
        if abs(self.angle_parcouru()) < abs(self.rotation_cible):
            msg.angular.z = 0.4 if self.rotation_cible > 0 else
-0.4
        else:
            msg.angular.z = 0.0
            self.etat = "AVANCER"

    self.pub.publish(msg)

```

3. Étape 3 : Suivi de Couloir Réactif, Régulateur PD et Lissage des Commandes (Exercice 5)

Pour conclure cette première phase d'expérimentation physique, le cinquième exercice introduit un comportement continu beaucoup plus fluide. Plutôt que de pivoter à vitesse fixe après l'arrêt du robot, ce script implémente une régulation dynamique basée sur un **correcteur Proportionnel-Dérivé (PD)**, une **anticipation géométrique des virages**, ainsi qu'un **filtre passe-bas logiciel** sur les commandes moteur.

Architecture de la Machine à États Hiérarchisée : La fonction `boucle()` évalue à chaque période dt l'état environnemental parmi 4 modes :

1. STOP : Arrêt immédiat si un obstacle frontal est détecté sous la distance minimale dist_stop .
2. PASSAGE_ETROIT : Activation lorsque deux murs sont détectés de chaque côté ($L_{\text{couloir}} < 1,80 \text{ m}$), imposant une vitesse réduite (40% de la vitesse max) et un centrage strict.
3. VIRAGE_INTERSECTION : Amorce préventive de courbe si la distance frontale passe sous 0,85 m.
4. SUIVI_MUR : Mode nominal avec ajustement dynamique de l'erreur latérale.

Calcul de la Régulation PD et Anticipation : L'erreur de centrage latérale $e_{\text{suivi}} = d_{\text{gauche}} - d_{\text{droite}}$ est pondérée par une composante d'anticipation e_{anti} analysant les zones diagonales avant-gauche et avant-droite ($\text{dist_devant_gauche}$, $\text{dist_devant_droite}$). La vitesse angulaire commandée $u_z(t)$ s'exprime par :

$$e_{\text{totale}}(t) = e_{\text{suivi}}(t) + e_{\text{anti}}(t)$$

$$u_z(t) = K_p \cdot e_{\text{totale}}(t) + K_d \cdot \frac{e_{\text{totale}}(t) - e_{\text{totale}}(t - dt)}{dt}$$

Atténuation des à-coups Moteur (Filtre Passe-Bas) : Pour éviter les variations brutales de couple sur les moteurs du Scout Mini et éliminer les secousses mécaniques, un filtre à réponse impulsionnelle infinie (lissage exponentiel) est appliqué sur les vitesses linéaire v_x et angulaire w_z avec un facteur $\alpha = \text{alpha_lissage}$:

$$v_{\text{filtré}}(t) = \alpha \cdot v_{\text{calculé}}(t) + (1 - \alpha) \cdot v_{\text{filtré}}(t - dt)$$

Listing 3.3 – Asservissement PD, anticipation et lissage des commandes (Code Ex 5).

```
# Extrait de la logique d'asservissement PD et lissage (Code 5)
derivee_angulaire = (erreur_totale - self.erreur_ang_precedente) /
    dt
self.erreur_ang_precedente = erreur_totale

# Calcul de la commande angulaire brute
cmd_z = (self.kp_ang * erreur_totale) + (self.kd_ang *
    derivee_angulaire)
cmd_z = max(-self.vitesse_ang_max, min(self.vitesse_ang_max, cmd_z)
    )

# Application du lissage exponentiel (filtre passe-bas)
self.cmd_x_filtree = (self.alpha_lissage * cmd_x) + ((1.0 - self.
    alpha_lissage) * self.cmd_x_filtree)
self.cmd_z_filtree = (self.alpha_lissage * cmd_z) + ((1.0 - self.
    alpha_lissage) * self.cmd_z_filtree)

msg.linear.x = self.cmd_x_filtree
msg.angular.z = self.cmd_z_filtree
self.pub.publish(msg)
```

4. Bilan de la Phase 1 et transition vers la vision

Cette première phase d'essais en salle a permis de valider :

- La réactivité du conteneur Docker et du bus de communication CAN du robot sous ROS 2.
- La plage de vitesses linéaires (0,2 m/s à 0,5 m/s) adaptées à l'environnement d'essai.
- L'efficacité du filtrage 3D du LiDAR Robosense pour la sécurité périphérique.
- La création complète du sujet et des scripts de TP réutilisables pour l'enseignement.

Une fois cette couche de contrôle bas niveau et de sécurité validée, les travaux se sont orientés vers l'objectif principal du projet : la navigation guidée par vision (suivi de ligne au sol) et la détection d'éléments de signalisation routière.

3.1.2 Phase 2 : Analyse Visuelle Avancée, Architecture Découplée et Régulation Adaptative

Après la validation des bases de commande et de sécurité LiDAR lors de la Phase 1, cette deuxième phase s'est concentrée sur la perception visuelle de la piste et l'optimisation des algorithmes de suivi de ligne.

Une démarche d'ingénierie itérative a été adoptée : tester des approches simples, analyser leurs défaillances face aux contraintes du monde réel (ombres, reflets, virages serrés, bifurcations), puis complexifier l'algorithme jusqu'à obtenir un comportement fluide et robuste.

1. Choix d'Architecture ROS 2 : Découplage Perception / Contrôle

Pour garantir une modularité maximale et faciliter le débogage, le système de suivi de ligne a été découpé en deux nœuds ROS 2 distincts communiquant par échanges de messages inter-nœuds :

1. **Le Nœud de Perception (`vision_line_node`)** : Il reçoit le flux RGB de la caméra, effectue les traitements OpenCV (filtrage, extraction de caractéristiques, calculs de masses), et publie l'erreur de centrage sur le topic `/line_tracking/error_raw` ainsi que l'état des zones sur `/camera/zones_route`.
2. **Le Nœud de Contrôle (`control_line_node`)** : Il s'abonne aux topics d'analyse visuelle, gère la mémoire de trajectoire, exécute le correcteur PD et calcule les vitesses linéaire v_x et angulaire w_z transmises au topic `/cmd_vel`.

2. Évolution du Nœud de Perception : Du Binarisme aux Scanlines Multi-Zonales

Le développement du nœud de vision s'est fait en trois itérations successives :

Itération 1 – Barycentre par Moments géométriques (Approche TP 2.1) : La première approche a consisté à isoler une Région d'Intérêt (ROI^*) en bas de l'image (lignes 420 à 480 sur une résolution 848×480) et à binariser l'image par un seuillage fixe (`cv2.threshold`). Le centre horizontal de la ligne C_x est obtenu par le rapport des moments d'ordre 1 et 0 [cite : 4] :

$$M_{00} = \sum_x \sum_y I(x, y), \quad M_{10} = \sum_x \sum_y x \cdot I(x, y) \implies C_x = \frac{M_{10}}{M_{00}}$$

L'erreur spatiale en pixels est ensuite calculée par rapport à l'axe optique central ($W/2 = 424 \text{ px}$) : $e = C_x - 424$ [cite : 4].

Analyse des limites : Cette méthode est très sensible au choix du drapeau de binarisation (THRESH_BINARY vs THRESH_BINARY_INV)[cite : 4]. Si la piste présente des lignes sombres sur sol clair ou inversement, l'inversion du drapeau est critique sous peine de calculer le centre du sol plutôt que celui de la ligne[cite : 4]. De plus, un simple binarisme ne résiste pas aux ombres ni aux reflets du sol éclairé.

Itération 2 – Partitionnement Multi-Lignes (L_1, L_2, L_3) et Secteurs (G, C, D) : Pour corriger l'incapacité à anticiper les virages, la ROI unique a été remplacée par un réseau de lignes de balayage horizontales (*scanlines*) placées à des distances de visée différentes ($Y_{L1} = 479 \text{ px}$, $Y_{L2} = 400 \text{ px}$, $Y_{L3} = 320 \text{ px}$), découpées en secteurs Gauche, Centre et Droite. L'erreur globale transmise au contrôleur combine la position immédiate (L_1) et la visée plus éloignée (L_2) selon une pondération géométrique :

$$e_{\text{calculée}} = 0,8 \cdot e_{L1} + 0,2 \cdot e_{L2}$$

Si la ligne immédiate L_1 est masquée (ombre localisée), le nœud bascule temporairement à 100% sur L_2 , évitant ainsi la perte de suivi.

Itération 3 – Filtrage HSV Dynamique, Morphologie et Étalonnage Autonome : Pour éliminer les reflets du sol de la salle de TP, un filtrage dans l'espace colorimétrique HSV a été combiné à une analyse de forme géométrique par contours :

- **Filtrage dynamique de la Saturation (S) :** Calcul de la saturation médiane des pixels cibles pour ajuster le seuil bas face aux variations de lumière.
- **Filtrage par ratio d'aspect :** Détection des contours et vérification du ratio longueur/largeur ($\frac{\max(w,h)}{\min(w,h)} \geq 3$). Une vraie ligne de guidage étant élançée, ce filtre élimine efficacement les reflets spéculaires arrondis.
- **Auto-calibration de la largeur de voie :** Quand les 2 bordures de voie sont visibles sur L_1 , le nœud calcule et lisse la largeur apparente de la route en pixels via une moyenne exponentielle :

$$W_{\text{voie}}(k) = 0,7 \cdot W_{\text{voie}}(k-1) + 0,3 \cdot |x_{\text{droite}} - x_{\text{gauche}}|$$

Si une seule bordure reste visible (ex : virage très serré), le robot reconstruit le centre théorique de la chaussée en appliquant un décalage de $\pm \frac{W_{\text{voie}}}{2}$.

3. Évolution du Nœud de Contrôle : Régulation PD, Mémoire et Anticipation

Le nœud de commande a évolué en parallèle pour passer d'un simple correcteur P saccadé[cite : 4] à un asservissement dynamique évolué.

Correcteur PD à Pas de Temps Dynamique (dt) : Conformément aux analyses menées dans le TP 2[cite : 4], la commande angulaire intègre une composante dérivée calculée avec le delta de temps réel de la boucle ROS 2 (calculé via `get_clock().now()`)[cite : 4] :

$$u_z(t) = - \left(K_p \cdot e(t) + K_d \cdot \frac{e(t) - e(t-dt)}{dt} \right)$$

La prise en compte de la dérivée $\frac{de}{dt}$ permet d'injecter un couple de braquage immédiat dès l'amorce d'une courbe, avant même que l'erreur proportionnelle $e(t)$ ne devienne critique[cite : 4].

Réduction Adaptative de la Vitesse Linéaire : Pour éviter que le robot ne dérive vers l'extérieur dans les virages en épingle, le nœud maintient une fenêtre glissante sur 1 seconde (10 échantillons à 10 Hz) récapitulant l'angle total de manœuvre exécuté $\theta_{\text{cumul}} = \sum |\omega_z \cdot dt|$. Lorsque θ_{cumul} dépasse un seuil critique (20°), la vitesse linéaire nominale ($V_{\text{MAX}} = 0,4 \text{ m/s}$) est automatiquement réduite via une interpolation linéaire vers une vitesse plancher ($V_{\text{MIN}} = 0,10 \text{ m/s}$), garantissant une adhérence parfaite sans risquer l'arrêt complet.

Gestion des Bifurcations en Y et Perte de Piste : Le nœud intègre une mémoire d'orientation long terme (`dernier_sens_virage`). Lors de l'approche d'une bifurcation détectée au loin par la ligne L_3 (L_{3G} et L_{3D} simultanément actifs), le robot applique un léger biais d'anticipation dans la direction privilégiée du circuit. Si la piste est intégralement perdue, le robot pivote sur lui-même du côté de son dernier virage enregistré pour retrouver la ligne.

4. Synthèse Pédagogique (TP 2) et Transition Matérielle vers la Phase 3

Transposition Pédagogique (TP 2) : Bien que les algorithmes finaux développés ci-dessus intègrent une auto-calibration et des filtres complexes, le document pédagogique rédigé pour les étudiants (*TP 2 : Guidage par Vision Artificielle et Intégration Système*)[cite : 4] a sciemment conservé une structure plus accessible :

- Focus sur la binarisation (`cv2.threshold`), le rôle physique du rapport $\frac{M_{10}}{M_{00}}$ [cite : 4] et le découplage des gains ROS 2 (K_p , K_d) dans la portée du nœud [cite : 4].
- Introduction d'une Machine à États Finis (FSM) à 3 modes principaux (`LIGNE_DROITE`, `VIRAGE_BRUSQUE`, `DEMI_TOUR`)[cite : 4], idéale pour faire assimiler la logique de contrôle hybride aux étudiants sans les charger de détails d'analyse d'image trop avancés.

Limitation Matérielle et Décision de Changement de Capteur : Au terme de cette Phase 2, malgré d'excellents résultats de suivi sur circuit, une contrainte matérielle majeure est apparue. La caméra Intel RealSense D435 installée initialement présentait un champ de vision (*FOV*) trop restreint en largeur, combiné à un positionnement en hauteur fixe non orientable. Cette géométrie créait une zone aveugle importante au pied du robot ($> 82 \text{ cm}$), rendant la détection des lignes très serrées délicate et imposant une visée artificielle très basse dans l'image.

Afin d'aborder la Phase 3 (intégration finale, franchissement d'intersections complexes et perception simultanée des panneaux), il a été décidé de **remplacer la caméra RealSense par une caméra Logitech C505e**. Ce capteur, réinstallé plus bas sur le châssis et doté d'une inclinaison mécanique ajustable, offre un champ de vision au sol beaucoup plus large et adapté au suivi de trajectoire rapproché.

3.1.3 Synthèse de la Phase 3 : Fusion Perception, Contrôle & Sécurité

La Phase 3 marque l'aboutissement de la chaîne de commande autonome du robot. Elle regroupe la perception visuelle multi-horizon, le contrôle de trajectoire intelligent et une couche de sécurité matérielle 3D.

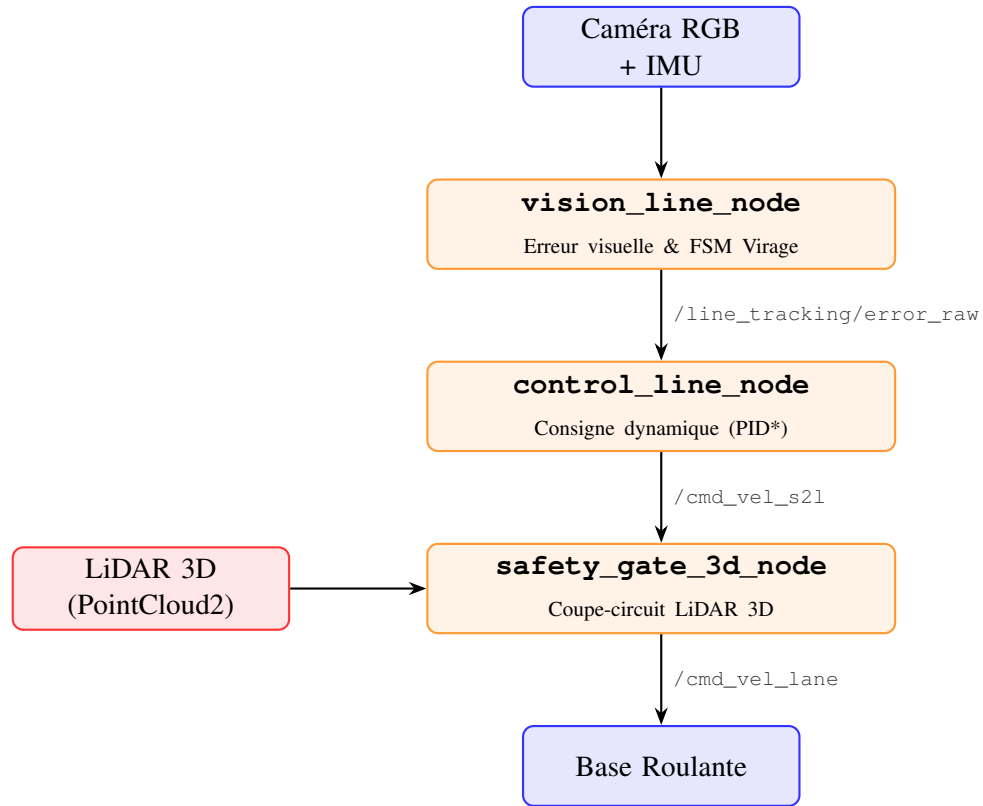


FIGURE 3.1 – Architecture de flux ROS 2 et chaîne de commande intégrée (Phase 3).

3.1.4 1. Nœud de Perception & FSM : vision_line_node

Ce nœud transforme le flux vidéo brut en une donnée d'écart géométrique exploitable, assisté par l'unité de mesure inertielle (IMU) pour la gestion des manœuvres complexes.

- **Traitement visuel multi-horizons** : Analyse simultanée de trois lignes de visée (L_1 bas, L_2 milieu, L_3 haut) pour anticiper la géométrie de la piste.
- **Persistance spatio-temporelle** : Utilisation d'un historique de masques (std::deque de 5 images) pour éliminer le bruit lumineux et les faux positifs.
- **Machine à États Finis (FSM) & Gyroscope**
 - Gestion autonome des virages serrés (ATTENTE_VIRAGE → APPROCHE_VIRAGE → FORCAGE_DROITE).
 - Intégration en temps réel de la vitesse angulaire ω_z transmise par l'IMU afin de calculer l'angle réel parcouru :

$$\theta(t) = \int_0^t |\omega_z(\tau)| d\tau \quad (3.1)$$

Cette valeur permet de valider la sortie de virage dès que $\theta \geq 45^\circ$.

- **Mode dégradé** : Capacité à naviguer avec une seule ligne détectée en estimant le centre de la voie via la largeur mémorisée.

3.1.5 2. Nœud de Contrôle Dynamique : `control_line_node`

Ce nœud traduit les erreurs de perception en commandes de vitesses linéaire v_x et angulaire ω_z adaptées au contexte.

- **Correction PID dynamique** : Calcule en continu la vitesse de rotation en fonction de l'erreur $e(t)$:

$$u_r(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt} \quad (3.2)$$

La vitesse linéaire v_x est automatiquement ajustée de manière inversement proportionnelle à la sévérité de l'erreur.

- **Adaptation environnementale (Secours Solaire)** : Ajustement asynchrone de l'exposition de la caméra RealSense via les services ROS 2 pour maintenir un contraste optimal en cas d'éblouissement.
- **Gestion multi-modes** :
 - **NOMINAL** : Conduite fluide avec ralentissement automatique en courbe.
 - **EPINGLE / DETRESSE** : Manœuvres de pivot sur place si la piste est temporairement perdue.
 - **BIFURCATION** : Biais de trajectoire basé sur la mémoire géométrique du dernier virage.

3.1.6 3. Filtre de Sécurité 3D : `safety_gate_3d_node`

Il agit comme un **multiplexeur de sécurité matériel** (*Supervisor / Safety Interlock*).

- **Zone de Sécurité Bounding Box 3D** : Filtrage géométrique strict du nuage de points (`sensor_msgs/msg/PointCloud2`) en dimensions réelles :

$$x \in [0.1 \text{ m}, 0.5 \text{ m}] \quad (\text{Distance avant})$$

$$y \in [-0.35 \text{ m}, 0.35 \text{ m}] \quad (\text{Largeur du corridor})$$

$$z \in [-0.20 \text{ m}, 0.50 \text{ m}] \quad (\text{Hauteur utile, hors sol})$$

- **Interception des commandes** : Si le nombre de points détectés dans ce volume dépasse le seuil de bruit toléré, la consigne venant du contrôleur (`/cmd_vel_s2l`) est immédiatement interceptée et forcée à $(0, 0)$ sur le topic de sortie (`/cmd_vel_lane`).

3.1.7 Lien avec l'Architecture Globale du Projet

Cette Phase 3 représente le **cœur décisionnel et exécutif** du robot autonome :

1. **Complémentarité des capteurs (Fusion Multi-Capteurs)** :
2. La **caméra RGB** fournit la résolution spatiale nécessaire au *suivi de marquage au sol*.
3. L'**IMU** offre une mesure cinématique rapide, insensible aux aléas lumineux, idéale pour *valider une rotation réelle*.
4. Le **LiDAR 3D** garantit la sécurité spatiale absolue en tant que *garde-fou physique*.
5. **Fiabilité et Hiérarchie de Sécurité** : En découplant le suivi de ligne (logiciel) de l'arrêt d'urgence (sécurité 3D), le système respecte le principe fondamental de la robotique autonome : **la sécurité matérielle prévaut toujours sur la navigation**.

6. **Modularité ROS 2** : Chaque nœud assure une responsabilité unique (Percevoir → Décider → Sécuriser). La communication via les topics ROS 2 (`/line_tracking`, `/error_raw`, `/cmd_vel_s2l`, `/cmd_vel_lane`) facilite les tests unitaires et la traçabilité des données.

3.2 Aparte : Intégration Globale et Validation Système

Pour conclure cette partie, il est essentiel de replacer la **Phase 3** dans le contexte du projet global. La chaîne de navigation et de sécurité développée (Perception visuelle, Contrôle PID et Sécurité 3D) s'intègre au sein d'une architecture globale orchestrée par un **Manager Central** fonctionnant sous forme de Machine à États Finis (FSM).

1. Coordination par le Manager Global

Le Manager agit comme le cerveau haut niveau du robot. Il arbitre la conduite en interceptant et croisant deux flux d'informations principaux :

- **Le flux de suivi de voie (Module Navigation)** : Transmission de la consigne dynamique de vitesse et d'orientation (`/cmd_vel_lane`) issue de mon travail de fusion perception-contrôle.
- **Le flux de détection d'objets (Module IA Visuelle)** : Transmission des événements de signalisation (panneaux, feux, piétons) identifiés en temps réel par le réseau de neurones.

2. Validation Expérimentale et Comportements Robot

Lors des tests d'intégration en conditions réelles, le système global a démontré une excellente robustesse. Les consignes transmises par le module de suivi de voie ont permis au robot de maintenir une trajectoire fluide sans jamais dévier de la route, tout en respectant scrupuleusement la logique de circulation imposée par le Manager :

- **Maintien de voie nominal** : Suivi rigoureux du marquage au sol, y compris dans les enchaînements de virages serrés.
- **Panneau STOP** : À la détection du panneau par l'IA, le robot marque un arrêt complet pendant exactement 2 secondes avant de reprendre automatiquement sa navigation.
- **Feu de signalisation** : Le robot s'immobilise immédiatement à l'apparition d'un feu rouge et reste à l'arrêt tant que le feu ne passe pas au vert.
- **Passage Piéton** : Détection et arrêt immédiat face à un piéton engagé, le robot attendant la libération totale de la voie pour redémarrer.
- **Barrière de Sécurité 3D (`safety_gate`)** : En parallèle des décisions du Manager, si un obstacle imprévu pénètre dans le volume de sécurité 3D du LiDAR, le système coupe immédiatement la vitesse, primant sur tout autre ordre de conduite.

3. Bilan de la Collaboration

Cette architecture modulaire a prouvé son efficacité : le découpage propre entre la navigation continue bas niveau, la prise de décision haut niveau basée sur l'IA et le coupe-circuit matériel 3D garantit un déplacement autonome fluide, sûr et parfaitement conforme au cahier des charges.

Chapitre 4

Contributions Transversales, Documentation & Environnement de Test

Outre le développement du cœur de contrôle et de sécurité (Phase 3), plusieurs tâches transversales, logistiques et documentaires ont été menées pour garantir la continuité du projet et la mise en valeur des robots auprès de l'école.

4.0.1 1. Rédaction de la Documentation Technique : Dépannage Docker

Au cours du développement, la modification du conteneur Docker ou l'ajout de nouvelles dépendances système ont fréquemment causé des instabilités (perte d'affichage sous noVNC/RViz, arrêt de publication des topics ROS 2, saturation mémoire).

Afin d'éviter des blocages prolongés pour l'équipe, un document de référence (*DOCKER DEBUG*) a été spécialement rédigé et mis à disposition. Il synthétise une procédure pas-à-pas pour restaurer rapidement l'environnement :

- **Gestion du Serveur d’Affichage** : Résolution des erreurs liées à `xhost` et `DISPLAY` pour rétablir le flux visuel noVNC :

```
export DISPLAY=:1  
./build_and_run.sh
```

- **Procédure de Réinitialisation Propre** : Méthode séquentielle de nettoyage complet du workspace (`build/`, `install/`, `log/`), re-sourcing du noyau ROS 2 Humble et mise à jour dynamique des dépendances via `rosdep`.

- **Compilation Contrainte (Gestion Mémoire)** : Limitation explicite du processus de compilation à un seul thread pour éviter la saturation de la mémoire vive embarquée du robot :

```
colcon build-symlink-install --cmake-args  
-DCMAKE_BUILD_TYPE=Release \  
--parallel-workers 1
```

4.0.2 2. Exploration Fonctionnelle & Préparation du Circuit d’Essai

- **Exploration du Module IA** : Durant plusieurs jours, une phase de recherche et de premiers essais sur la détection par IA a été menée, avant de passer le relais à mon collègue Maxime Delubac qui a finalisé ce composant.
- **Tracé et Modélisation du Circuit** : Conception et réalisation physique de la route à l'aide de bandes adhésives au sol, afin de définir un parcours d'évaluation représentatif (lignes droites, courbes, intersections, zones d'arrêt).

4.0.3 3. Évaluation en Conditions Réelles & Contraintes Environnementales

Le projet visait initialement une démonstration finale dans une salle dédiée, devant un parterre de professeurs et le directeur de Polytech Nice-Sophia, afin de prouver concrètement le retour sur investissement réalisé dans ces plateformes robotiques. N'ayant finalement pas pu accéder à cette salle adjacente, la présentation et les essais ont dû se dérouler dans la salle de test habituelle.

Cette contrainte de dernière minute a exposé le système de vision à des conditions particulièrement exigeantes :

- **Éclairage Extrême** : En raison du climat ensoleillé de Valbonne et de la présence de grandes baies vitrées, la lumière naturelle directe variait fortement au cours de la journée.
- **Revêtement Complexe** : Le sol sombre et fortement réfléchissant générait des réflexions spéculaires parasites sur le ruban de la piste.
- **Perturbations Visuelles Visuelles** : La présence de pieds de tables blancs immobiles créait de faux positifs potentiels pour le suivi de ligne.

Remarque : Ces contraintes environnementales sévères ont finalement servi de crash-test grandeur nature, démontrant la viabilité de la persistance spatio-temporelle et du secours solaire intégrés dans le module de perception.

4.0.4 4. Proximité et Synergie avec le FabLab

La proximité immédiate du FabLab de l'école a constitué un atout logistique majeur pour l'équipe, permettant le prototypage rapide d'outillage et de supports physiques :

- Impression 3D d'un support sur-mesure pour la caméra Logitech (réalisée par un collègue du groupe).
- Fabrication 2D/3D du mannequin d'essai (piéton) servant aux tests d'arrêt d'urgence par l'IA et le LiDAR.
- Accès continu à l'outillage pour l'ajustement mécanique et le câblage sur le robot.

Chapitre 5

Conclusion et Perspectives

5.1 Conclusion Générale

Ce projet a permis de concevoir, valider et déployer une architecture de navigation autonome complète, hautement modulaire et sécurisée sur le robot mobile Scout Mini sous ROS 2 Humble. À travers une démarche d'ingénierie progressive, les trois phases clés du projet ont apporté des réponses concrètes aux défis de la robotique autonome :

1. **Maîtrise de l'Infrastructure et du SLAM 3D** : La configuration contrôlée de l'environnement Docker et l'optimisation des cartes de coût Nav2 ont permis d'exploiter la carte 3D générée par **RTAB-Map** et l'odométrie ICP du LiDAR Robosense 16P, garantissant des déplacements fiables dans des espaces restreints.
2. **Perception Visuelle et Régulation Dynamique** : Le suivi de ligne a évolué d'un simple calcul de moments géométriques vers une analyse visuelle multi-horizons (L_1, L_2, L_3) associée à un filtrage HSV dynamique et à une calibration automatique de la voie. L'intégration du correcteur PD adaptatif ajustant la vitesse selon la courbure a offert un comportement fluide sur piste.
3. **Fusion Multi-Capteurs, Sécurité 3D & Manager Global** : L'association de l'IMU (validation gyroscopique des virages), du suivi visuel et du filtre de sécurité 3D LiDAR (`safety_gate_3d_node`) a constitué un socle d'exécution robuste. Intégré au Manager central FSM, le robot a démontré un respect parfait du code de la route (arrêt STOP de 2s, feux rouges, passage piéton) et un freinage d'urgence autonome en cas d'obstacle imprévu.

Enfin, la rédaction du guide technique *DOCKER DEBUG* et l'aménagement physique du circuit d'essai ont assuré la transmission du savoir-faire et la pérennité du projet au sein de l'école.

5.2 Perspectives et Évolutions Futures

Plusieurs pistes de développement peuvent être envisagées pour prolonger ces travaux :

1. Valorisations Applicatives

- **Robots de Service et Logistique d'Intérieur** : Adaptation de la brique de suivi de ligne et de sécurité 3D pour des robots de livraison de documents en bureau ou de nettoyage industriel nécessitant un suivi de trajectoire strict.

- **Robots d'Exploration en Milieu Hostile** : Utilisation de la stack RTAB-Map développée au Chapitre 1 pour des missions d'inspection dans des caves, galeries ou zones sans signal GPS, où la sécurité 3D LiDAR devient indispensable.

2. Optimisations Techniques et Algorithmiques

- **Projection 3D de la Route** : Exploiter la carte 3D pour projeter le marquage au sol dans le repère monde 3D, permettant d'anticiper la trajectoire à plus longue distance et de calculer une vitesse optimale en courbe.
- **Gain de Fréquence et Performance** : Optimiser la boucle d'analyse OpenCV (ex : passage sous C++ natif ou accélération matérielle) pour augmenter le nombre d'images par seconde (FPS) et permettre une conduite plus rapide.

3. Amélioration de l'Environnement d'Évaluation

- **Optimisation du Revêtement** : Remplacer le sol réfléchissant de la salle d'essai par un revêtement mat sombre dédié, éliminant les reflets spéculaires et maximisant le contraste du ruban de guidage.

Annexes

Annexe A : Fiche Technique — Dépannage Docker

Ce document récapitule la procédure de restauration d'urgence de l'environnement conteneurisé développée au cours du projet suite aux instabilités système.

Remarque : La fiche complète d'une page au format PDF (`DOCKER_DEBUG.pdf`) est jointe au dossier ou consultable sur le dépôt du projet.

1. Problème d'affichage graphique (`xhost` / `noVNC`)

Si le flux RViz ou `noVNC` ne s'affiche plus lors du lancement du conteneur :

```
# Sur la machine hte :  
echo $DISPLAY  
export DISPLAY=:1  
./build_and_run.sh  
  
# Si le problme persiste l'intérieur du conteneur :  
export DISPLAY=:1
```

2. Réinitialisation complète du Workspace ROS 2

En cas de corruption du build ou d'échec de compilation des packages :

```
# 1. Sourcing de la distribution officielle  
source /opt/ros/humble/setup.bash  
  
# 2. Nettoyage des dossiers gnrs  
rm -rf build/ install/ log/  
  
# 3. Mise jour des dpendances via rosdep  
sudo apt update  
cd /roskit_ws  
rosdep update  
rosdep install --from-paths src --ignore-src -r -y  
  
# 4. Compilation bride 1 thread (gestion de la mmoire RAM embarque  
 )  
colcon build-symlink-install --cmake-args -DCMAKE_BUILD_TYPE=  
Release \  
--parallel-workers 1  
  
# 5. Sourcing final  
source install/setup.sh
```

Annexe B : Dépôt de Code Source & Ressources (GitHub)

L'ensemble du code source développé (nœuds ROS 2, scripts d'analyse visuelle, fichiers de configuration des cartes de coût Nav2, ainsi que l'intégralité des 5 sujets et corrigés de TP) est hébergé et maintenu sur le dépôt GitHub du projet.

Lien du dépôt : <https://github.com/votre-compte/projet-scout-ros2>

Arborescence du Dépôt :

```
projet-scout-ros2/  
  src/  
    gr_bringup/           # Lancement hardware & capteurs  
    vision_line_node/     # Perception visuelle & FSM (Phase 3)  
    control_line_node/    # Régulateur PID & gestion vitesse  
    safety_gate_3d_node/  # Filtre de sécurité LiDAR 3D  
  docs/  
    DOCKER_DEBUG.pdf      # Guide de dépannage  
    TP_ROS2/              # Enoncés et corrigés des TP 1 à 5  
  README.md              # Instructions d'installation et lancement
```

Résumé / Abstract

Synthèse en français :

Ce rapport retrace la conception, le développement et le déploiement d'une chaîne de navigation autonome complète pour le robot mobile AgileX Scout Mini sous **ROS 2 Humble** et environnement conteneurisé **Docker**. Le rapport s'articule autour de trois axes majeurs :

- **Cartographie et SLAM** : Validation de l'infrastructure logicielle et déploiement du SLAM 2D (`slam_toolbox`) et 3D (**RTAB-Map**) avec odométrie LiDAR ICP et cartes de coût tridimensionnelles sous Nav2.
- **Développement Pédagogique et Suivi Visuel** : Conception de 3 Travaux Pratiques progressifs et création d'un système de suivi de ligne visuel découplé (Perception/Contrôle) avec régulation PD et filtres HSV adaptatifs.
- **Fusion Multi-Capteurs et Sécurité 3D** : Implémentation d'une architecture multi-horizons associant la caméra, l'IMU (validation gyroscopique des virages) et un coupe-circuit matériel 3D LiDAR (`safety_gate_3d_node`).

Interfacée avec un Manager IA orchestrant la signalisation routière (STOP, feux, piétons), la plateforme a démontré une grande fiabilité en conditions réelles, soutenue par une documentation de dépannage (*DOCKER DEBUG*) garantissant la pérennité des travaux.

Abstract in English :

*This report details the design, development, and deployment of an end-to-end autonomous navigation architecture for the AgileX Scout Mini mobile platform operating under **ROS 2 Humble** within a containerized **Docker** environment. The report is structured around three core pillars :*

- **SLAM & Mapping** : Implementation of software infrastructure and deployment of 2D (`slam_toolbox`) and 3D (**RTAB-Map**) SLAM incorporating LiDAR ICP odometry and 3D costmaps within Nav2.
- **Educational Framework & Visual Tracking** : Creation of 5 progressive hands-on lab sessions and engineering of a decoupled visual line-following pipeline featuring PD control and adaptive HSV filtering.
- **Multi-Sensor Fusion & 3D Safety Gate** : Integration of a multi-horizon control architecture combining camera vision, IMU (gyroscopic turn validation), and a 3D LiDAR safety interlock node (`safety_gate_3d_node`).

*Interfaced with a high-level AI Manager supervising traffic regulations (STOP signs, traffic lights, pedestrian crossings), the system demonstrated high reliability under challenging real-world environments, supported by detailed technical troubleshooting documentation (*DOCKER DEBUG*).*

Mots-clés : ROS 2 Humble, AgileX Scout Mini, Docker, SLAM 3D, RTAB-Map, Suivi de ligne, Contrôle PID, LiDAR 3D, Safety Interlock, Fusion Multi-Capteurs.

Keywords : ROS 2 Humble, AgileX Scout Mini, Docker, 3D SLAM, RTAB-Map, Line Tracking, PID Control, 3D LiDAR, Safety Interlock, Multi-Sensor Fusion.

Nom de l'étudiant : Thomas CHAVAROCHE

Titre du stage : [Le titre de ton stage]

Année d'exécution : [2026]

Structure d'accueil : [Polytech lab'] – [Adresse complète]

Responsable de stage : [M. Pascal MassonZ]